



Rapport Travaux Pratiques - Python



Réalisé par :

Omar Lamari & Alexis Paris

Année universitaire :

M1 EEA/TSI - 2025/2026

Enseignants responsables :

Mahdi Madani & Abdellah El-Zaar

TP 1 : La récursivité, le tri et la recherche dichotomique

But du TP :

L'objectif de ce TP est d'approfondir les notions fondamentales du langage Python (types, structures conditionnelles et répétitives) et de les appliquer à la conception d'algorithmes classiques en informatique : la récursivité à travers le calcul de la factorielle, de Fibonacci et du PGCD, la recherche efficace d'un élément dans une liste triée via la dichotomie, et le tri de données avec trois algorithmes distincts (tri à bulle, tri par sélection et tri par fusion).

Exercice 1 : Récursivité :

Factorielle récursive Fact(N) :

```
def Fact(N):  
    """Retourne la factorielle de N (entier naturel)."""  
    if N == 0 or N == 1:      # Cas de base  
        return 1  
    return N * Fact(N - 1)    # Cas récursif  
  
try:  
    N = int(input("Entrez un entier naturel N : "))  
    if N < 0:  
        print("Erreur : N doit être un entier naturel (>= 0).")  
    else:  
        print(f"Fact({N}) = {Fact(N)}")  
except ValueError:  
    print("Erreur : veuillez saisir un nombre entier.")
```

Cette fonction permet de calculer la factorielle d'un entier naturel saisi par l'utilisateur à son appel. Nous gérons tout d'abord le cas simple ou récursif, en appelant la fonction dans sa valeur de retour pour le calcul.

Nous avons rajouté des vérifications :

- si le nombre saisi est inférieur à 0, on renvoie une indication dans la console
- si le nombre saisi n'est pas un nombre entier exploitable (ValueError), alors on renvoie une erreur en console également

Résultat console :

```
Fact(3) = 6
```

```
Fact(21) = 51090942171709440000
```

Suite de Fibonacci Fibo(M) :

```
def Fibo(M):  
    """Retourne le M-ème terme de la suite de Fibonacci."""  
    if M == 0: # Cas de base : F(0) = 0  
        return 0  
    elif M == 1: # Cas de base : F(1) = 1  
        return 1  
    return Fibo(M - 1) + Fibo(M - 2) # Cas récursif  
  
try:  
    M = int(input("Entrez un entier naturel M : "))  
    if M < 0:  
        print("Erreur : M doit être un entier naturel (>= 0).")  
    else:  
        print(f"Fibo({M}) = {Fibo(M)}")  
except ValueError:  
    print("Erreur : veuillez saisir un nombre entier.")
```

Cette fonction quant à elle, permet de calculer les M premiers éléments de la suite de Fibonacci tout en retournant cet élément.

Trois cas sont alors proposés :

- Si M vaut 0, on retourne 0 (cas classique, pas de suite possible avec 0)
- Si M vaut 1, pareillement on retourne 1 (pas de suite réellement possible)
- Sinon on fait le calcul en faisant de la récursivité, on fait une addition de deux appels à fonction pour que le résultat se remplisse

La gestion d'erreur est la même que l'exemple précédent.

Résultat console :

```
Fibo(6) = 8
```

```
Fibo(20) = 6765
```

Remarque : Si l'on prend un entier M au dessus de 25, le programme a tendance à prendre énormément de temps

Plus grand diviseur commun (PGCD) :

```
def PGCD(A, B):  
    """Retourne le Plus Grand Commun Diviseur de A et B (A > B > 0)."""  
    if B == 0: # Cas de base : PGCD(A, 0) = A  
        return A  
    return PGCD(B, A % B) # Cas récursif : on remplace (A,B) par (B, A mod B)  
  
try:  
    A = int(input("Entrez le premier entier positif A : "))  
    B = int(input("Entrez le deuxième entier positif B (B < A) : "))  
    if A <= 0 or B <= 0:  
        print("Erreur : A et B doivent être des entiers strictement positifs (> 0)")  
    elif B >= A:  
        print(f"Erreur : A doit être strictement supérieur à B (A={A}, B={B}).")  
    else:  
        print(f"PGCD({A}, {B}) = {PGCD(A, B)}")  
except ValueError:  
    print("Erreur : veuillez saisir des nombres entiers.")
```

On s'attarde maintenant sur une fonction par appels récursifs qui permet de calculer le plus grand diviseur commun de deux entiers. Si le second entier soumis est égal à 0 on renvoie alors le premier proposé. Sinon on retourne simplement le PGCD.

Les valeurs sont saisies par un utilisateur, on a par la suite une vérification visant à déterminer si :

- Les deux nombres saisis sont des entiers positifs
- A est strictement supérieur à B
- Les valeurs sont bien des nombres entiers

Résultat console :

```
PGCD(10, 5) = 5
```

```
PGCD(268, 12) = 4
```

Exercice 2 : Recherche par dichotomie

```
def Dichotom(E, L):
    """Cherche E dans la liste triée L. Retourne E si trouvé, False sinon."""
    bas = 0 # Borne inférieure de la zone de recherche
    haut = len(L) - 1 # Borne supérieure de la zone de recherche

    while bas <= haut: # Tant que la zone n'est pas vide
        milieu = (bas + haut) // 2 # Indice central (division entière)

        if L[milieu] == E: # Trouvé
            return E
        elif E < L[milieu]: # E est dans la moitié gauche
            haut = milieu - 1
        else: # E est dans la moitié droite
            bas = milieu + 1

    return False # E absent de L

L = list(range(0, 51)) # Liste triée
print(f"Liste de recherche : {L}\n")

try:
    N = int(input("Entrez un nombre entier à rechercher dans la liste : "))
    resultat = Dichotom(N, L)
    if resultat is not False:
        print(f"Dichotom({N}, L) {resultat} Valeur trouvée dans la liste !")
    else:
        print(f"Dichotom({N}, L) False Valeur non trouvée dans la liste.")
except ValueError:
    print("Erreur : veuillez saisir un nombre entier.")
```

Il n'est ici plus réellement question de récursivité, cette fonction permet d'exécuter le procédé de dichotomie à l'aide de deux paramètres :

- Une saisie utilisateur stockée une fois indiquée dans la variable E
- Une liste L triée servant de référence pour la recherche

Résultat console :

```
Liste de recherche : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,
Dichotom(22, L) 22 Valeur trouvée dans la liste !
```

Exercice 3 : Les algorithmes de tri

Le Tri à bulle :

```
def Tri_Bulle(L):
    """Retourne une copie triée de L par l'algorithme du tri à bulle."""
    L_tri = L.copy() # On ne modifie pas la liste originale
    n = len(L_tri)

    for i in range(n - 1): # n-1 passes au maximum
        permutation = False # Drapeau : on se demande s'il y a permutation

        for j in range(n - 1 - i): # Les i derniers éléments sont déjà à leur place
            if L_tri[j] > L_tri[j + 1]:
                L_tri[j], L_tri[j + 1] = L_tri[j + 1], L_tri[j] # Échange
                permutation = True

        if not permutation: # si la liste est bien triée on stoppe
            break

    return L_tri

try:
    taille = int(input("Entrez la taille de la liste à trier (entre 2 et 40) : "))
    if taille < 2:
        print("Erreur : la taille doit être >= 2.")
    elif taille > 40:
        print("Erreur : la taille doit être <= 40.")
    else:
        L = random.sample(range(1, 41), taille)
        print(f"Avant : {L}")
        print(f"Après : {Tri_Bulle(L)}")
except ValueError:
    print("Erreur : veuillez saisir un nombre entier.")
```

Cette fonction compare des éléments d'une liste et elle fait basculer le plus grand élément de la liste comparée au plus à droite pour au final avoir un tri croissant (on permute à droite si plus grand d'un côté).

On demande à l'utilisateur la taille de la liste à trier au départ puis on vérifie si :

- c'est bien un entier supérieur à 2 et inférieur à 40
- c'est bien un nombre entier

Si tout se passe comme prévu, on stocke dans L une liste aléatoire d'un à 40 et de taille définie par l'utilisateur. Puis on affiche l'avant et l'après.

Affichage console :

```
Avant : [6, 9, 40, 36, 2, 35, 32, 8, 39, 38, 25, 23, 37, 17, 28]
Après : [2, 6, 8, 9, 17, 23, 25, 28, 32, 35, 36, 37, 38, 39, 40]
```

```
Avant : [29, 39, 38, 20]
Après : [20, 29, 38, 39]
```

Le tri par sélection :

```
def Tri_Select(L):
    """Retourne une copie triée de L par l'algorithme du tri par sélection."""
    L_tri = L.copy()
    n = len(L_tri)

    for i in range(n - 1): # i : début de la partie non triée
        indice_min = i # On suppose que L_tri[i] est le minimum
        for j in range(i + 1, n): # Recherche du vrai minimum dans l'intervalle
            if L_tri[j] < L_tri[indice_min]:
                indice_min = j

        if indice_min != i: # Permutation uniquement si nécessaire
            L_tri[i], L_tri[indice_min] = L_tri[indice_min], L_tri[i]

    return L_tri

try:
    taille = int(input("Entrez la taille de la liste à trier (entre 2 et 40) : "))
    if taille < 2:
        print("Erreur : la taille doit être >= 2.")
    elif taille > 40:
        print("Erreur : la taille doit être <= 40.")
    else:
        L = random.sample(range(1, 41), taille)
        print(f"Avant : {L}")
        print(f"Après : {Tri_Select(L)}")
except ValueError:
    print("Erreur : veuillez saisir un nombre entier.")
```

Cette méthode de tri cherche le minimum de la partie non triée et la place en tête de liste à chaque fois.

On impose comme l'exemple précédent un input utilisateur permettant de capter la taille de la liste et l'on revoit des erreurs si les cas d'erreur se présentent.

Résultat console :

```
Avant : [12, 11, 10, 21, 32, 6, 8, 23, 25, 9]
Après : [6, 8, 9, 10, 11, 12, 21, 23, 25, 32]
```

Similaire à avant, les résultats console ne donnent pas de variations, dans tous les cas la liste sera triée.

Le tri à fusion :

```
def _Fusionner(gauche, droite):
    """Fusionne deux listes triées en une seule liste triée."""
    resultat = []
    i = j = 0

    # On compare les têtes des deux listes et on insère le plus petit
    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i])
            i += 1
        else:
            resultat.append(droite[j])
            j += 1

    # On ajoute les éléments restants (une seule liste peut en avoir)
    resultat.extend(gauche[i:])
    resultat.extend(droite[j:])
    return resultat

def Tri_Fusion(L):
    """Retourne une copie triée de L par l'algorithme du tri par fusion (récursif)"""
    n = len(L)

    if n <= 1:
        # Cas de base : liste de 0 ou 1 élément déjà triée
        return L.copy()

    milieu = n // 2
    gauche = Tri_Fusion(L[:milieu])
    droite = Tri_Fusion(L[milieu:])

    return _Fusionner(gauche, droite) # Fusion des deux moitiés triées
```

Ce tri est divisé en deux fonctions distinctes :

- Fusionner : qui va permettre de fusionner deux listes triées et n'en faire qu'une tout cela en comparant les deux éléments en tête de liste à chaque étape
- Tri_Fusion : qui va permettre quant à elle de renvoyer une copie de la liste L grâce au final à la méthode Fusionner

```
try:
    taille = int(input("Entrez la taille de la liste à trier (entre 2 et 40) : "))
    if taille < 2:
        print("Erreur : la taille doit être >= 2.")
    elif taille > 40:
        print("Erreur : la taille doit être <= 40.")
    else:
        L = random.sample(range(1, 41), taille)
        print(f"Avant : {L}")
        print(f"Après : {Tri_Fusion(L)}")
except ValueError:
    print("Erreur : veuillez saisir un nombre entier.")
```

Résultat console :

```
Avant : [39, 33, 29, 8, 30, 34, 16, 1, 28, 7, 17, 37, 24, 27, 21, 2, 38]
Après : [1, 2, 7, 8, 16, 17, 21, 24, 27, 28, 29, 30, 33, 34, 37, 38, 39]
```

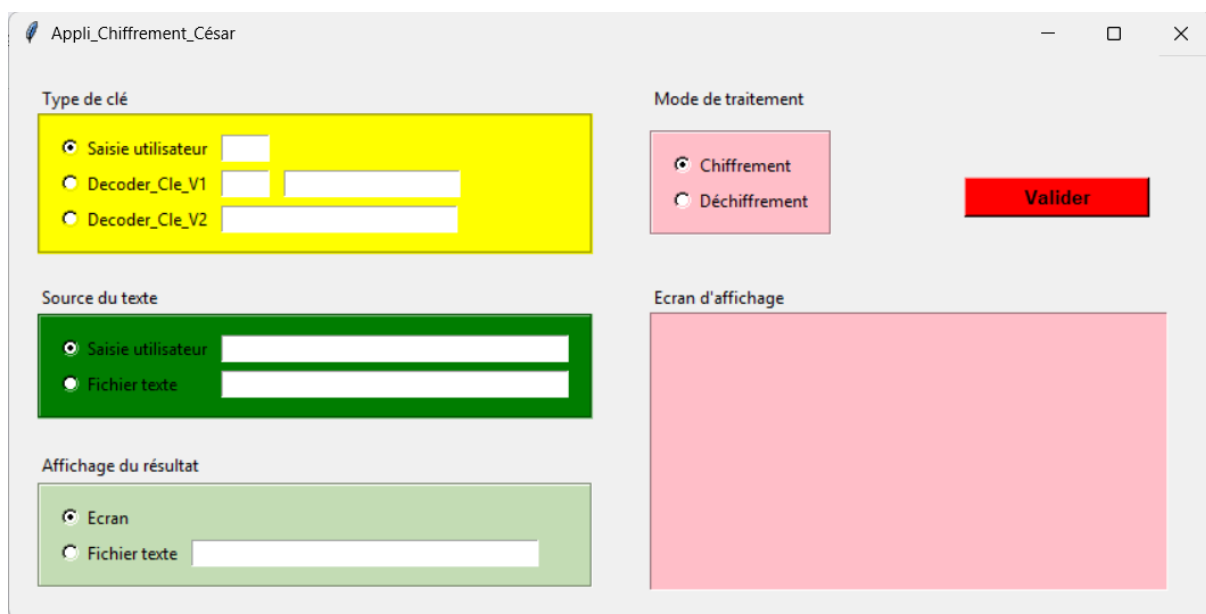
Aucune réelle différence encore.

TP 2 : Application de chiffrement César

But du TP :

L'objectif de ce TP est d'implémenter en Python le chiffrement de César, une technique cryptographique par décalage alphabétique. Cela implique de concevoir des fonctions permettant de chiffrer et de déchiffrer un texte à partir d'une clé donnée, mais aussi de retrouver automatiquement cette clé à partir d'informations partielles sur le message clair. Une interface graphique Tkinter est ensuite développée pour regrouper ces fonctionnalités dans une application interactive.

Voici une capture de l'interface :



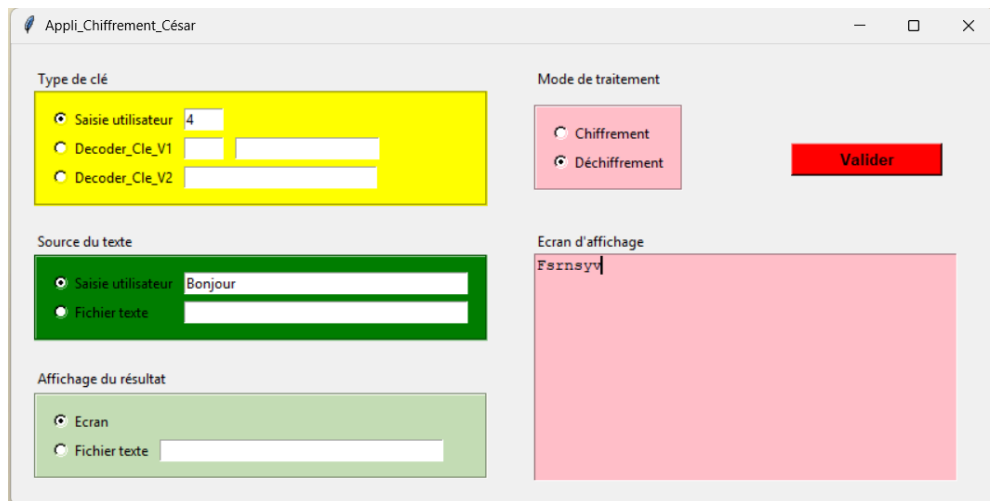
Cette petite application met en action les fonctions développées précédemment dans le TP. L'application se désigne en 2 parties :

- Une partie dite “métier” qui recense les fonctions utilisées à l'appel d'un bouton et un test de celles-ci dans un main
- Une partie dite “IHM” qui contient des blocs réalisés à l'aide de tkinter

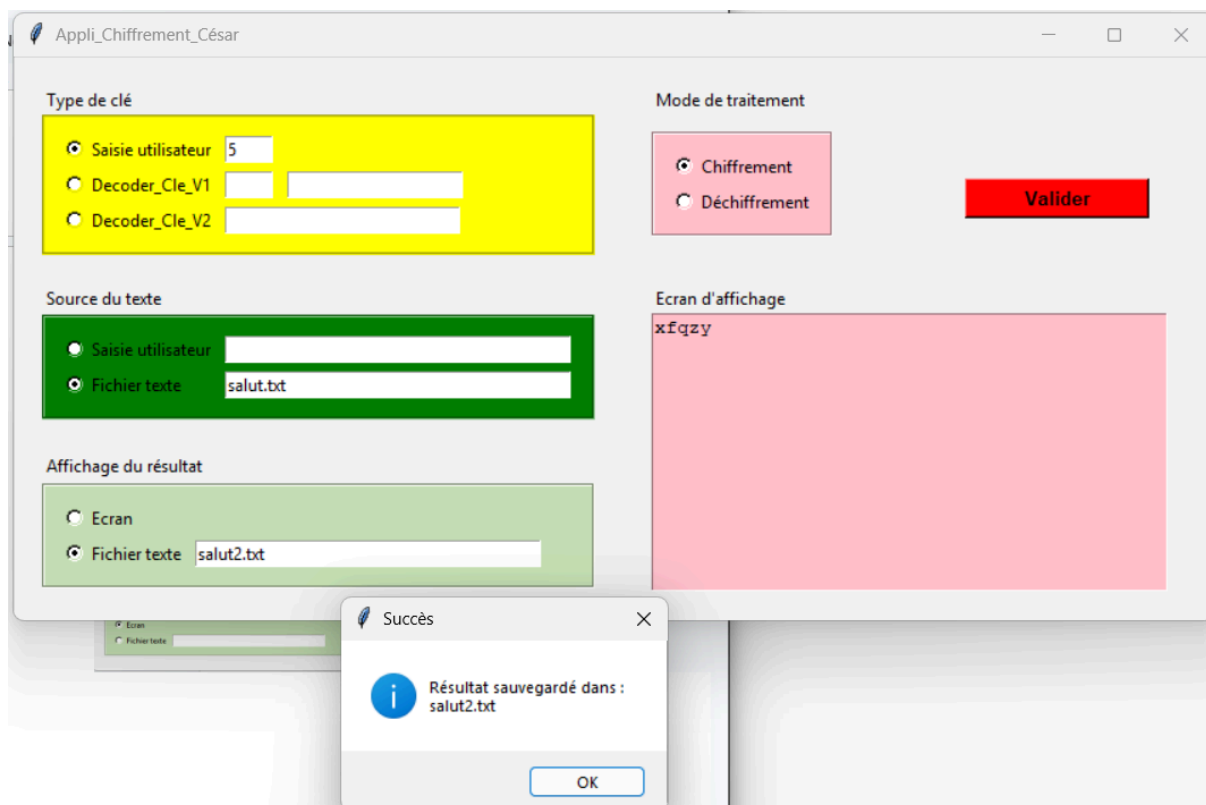
La partie métier est essentielle à l'IHM pour fonctionner et réaliser les opérations spécifiques que l'on lui demande.

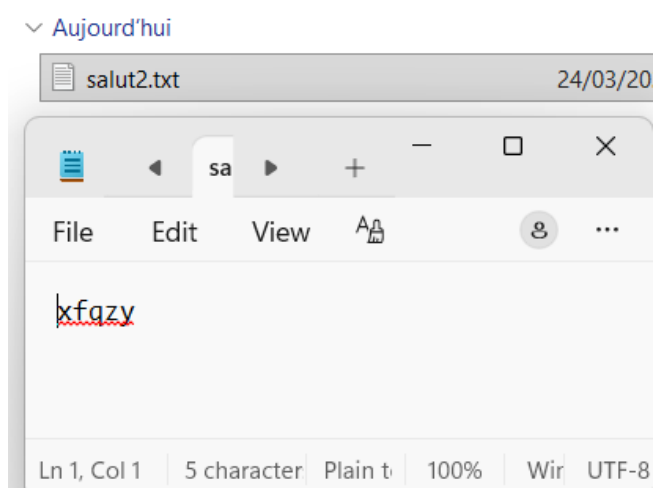
Exemple de fonctionnement :

Chiffrement avec un décalage de 4 de Bonjour :



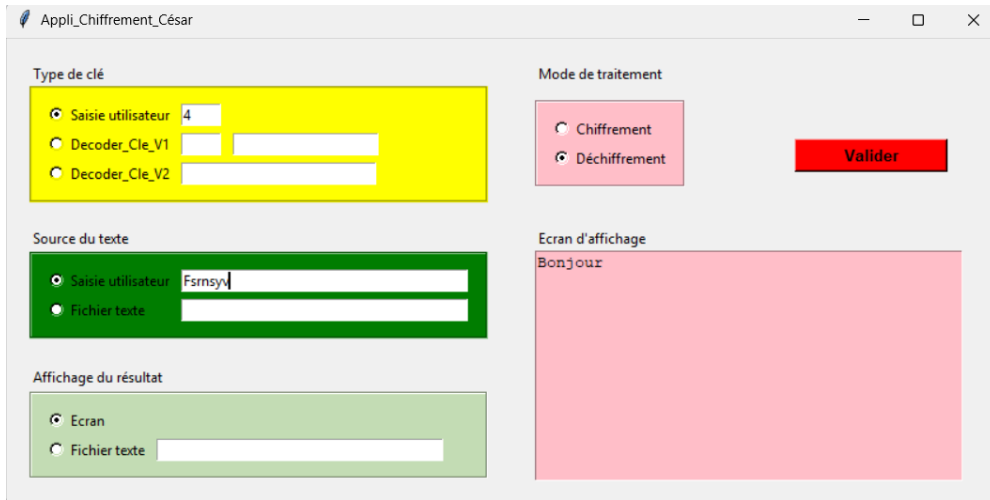
Chiffage de “Salut” avec un décalage de 5 depuis un fichier texte puis import dans un autre fichier :





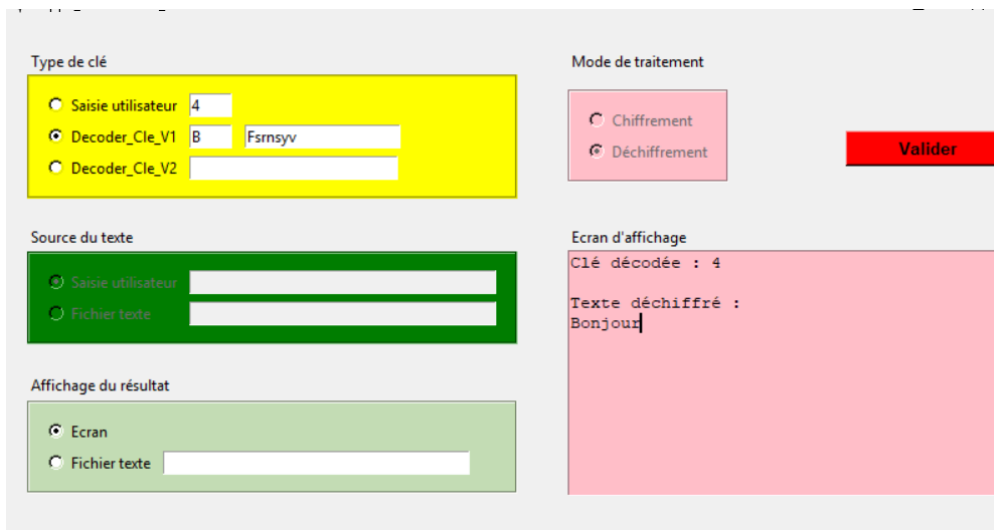
Le résultat présent dans l'écran d'affichage a pu être importé dans un fichier.

Déchiffrage de “Fsrsnsyv” avec renseignement du décalage 4 :



The screenshot shows the 'Appli_Chiffrement_César' window. The 'Type de clé' section has 'Saisie utilisateur' selected with a value of 4. The 'Mode de traitement' section has 'Déchiffrement' selected. The 'Source du texte' section has 'Saisie utilisateur' selected with the text 'Fsrsnsyv'. The 'Affichage du résultat' section has 'Ecran' selected. The 'Ecran d'affichage' shows the text 'Bonjour'. A red 'Valider' button is visible.

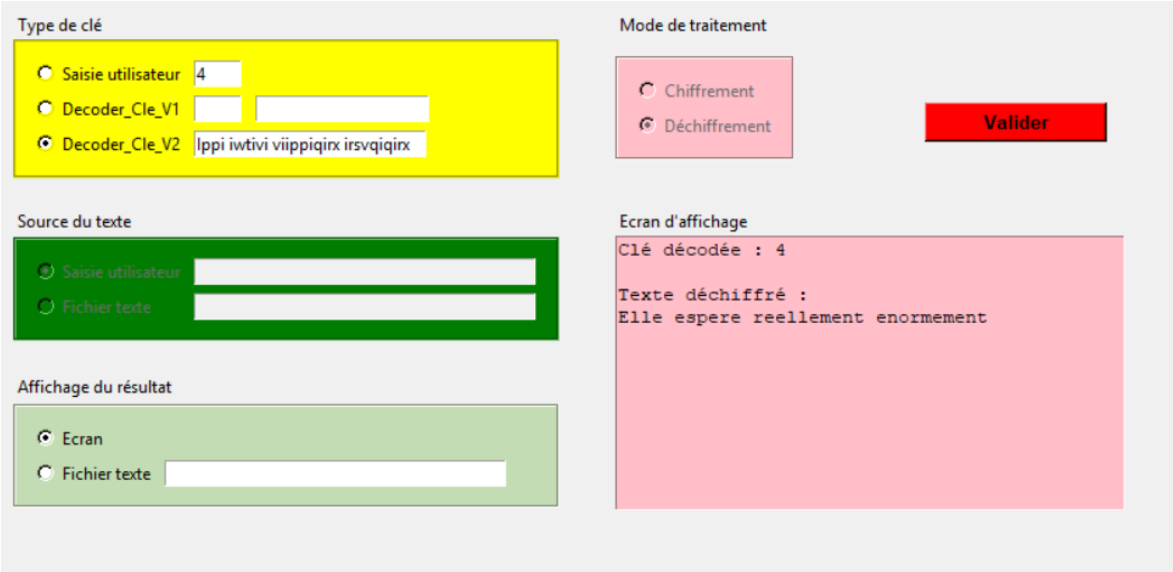
Essai de décodage clé V1 (renvoie la clé de chiffrement initiale) :



The screenshot shows the 'Appli_Chiffrement_César' window after clicking 'Valider'. The 'Type de clé' section now has 'Decoder_Cle_V1' selected with a value of B, and the 'Saisie utilisateur' field contains 'Fsrsnsyv'. The 'Mode de traitement' section remains 'Déchiffrement'. The 'Source du texte' section remains 'Saisie utilisateur'. The 'Affichage du résultat' section remains 'Ecran'. The 'Ecran d'affichage' now displays 'Clé décodée : 4' and 'Texte déchiffré : Bonjour'.

Remarque : Les champs de chiffrement et déchiffrement sont grisés car les fonctions de décodage ne les nécessitent pas.

Essai décodage V2 (très imprécis car il se base sur les “e”) :



Type de clé

☐ Saisie utilisateur 4

☐ Decoder_Cle_V1

☒ Decoder_Cle_V2 lppi iwtivi viippiqirx irsvqirx

Mode de traitement

☐ Chiffrement

☒ Déchiffrement

Valider

Source du texte

☒ Saisie utilisateur

☐ Fichier texte

Affichage du résultat

☒ Ecran

☐ Fichier texte

Ecran d'affichage

Clé décodée : 4

Texte déchiffré :
Elle espere reellement enormement

La phrase chiffrée a été volontairement choisie avec beaucoup de “e” pour qu’il y ait plus de chances que le message original soit retrouvé (chiffrée avec une clé de 4 à la base).

TP 3 : Programmation orientée objet

But du TP :

L'objectif de ce TP est de maîtriser les concepts clés de la programmation orientée objets en Python : la définition de classes avec leurs attributs et méthodes, l'encapsulation, l'héritage simple et l'héritage multiple. Pour cela, on modélise progressivement un ensemble de classes représentant des personnages à différentes étapes de leur vie (Personne, Etudiant, Cadre, Fonctionnaire, Retraité), en s'appuyant sur un diagramme UML fourni.

Exercice 1 : Organiser et encapsuler son code en classes et méthodes

```
[1]: class Personne:
    """Représente un personnage positionné dans un espace 2D."""

    def __init__(self):
        # Attributs par défaut : nom et position initiale
        self.nom = "Jule"
        self.x = 1
        self.y = 3

    def __repr__(self):
        """Affichage : nom et position [x, y]."""
        return f"{self.nom} est à la position [{self.x}, {self.y}]"

    def avancer_d(self, pas):
        """Déplacement vers la droite (+x)."""
        self.x += pas

    def avancer_g(self, pas):
        """Déplacement vers la gauche (-x)."""
        self.x -= pas

    def avancer_h(self, pas):
        """Déplacement vers le haut (+y)."""
        self.y += pas

    def avancer_b(self, pas):
        """Déplacement vers le bas (-y)."""
        self.y -= pas
```

Explication : La classe `Personne` modélise un personnage avec un nom et une position (x, y) dans un espace 2D. Le constructeur initialise ces attributs. La méthode `__repr__` retourne une représentation lisible de l'objet. Les quatre méthodes de déplacement (`avancer_d`, `avancer_g`, `avancer_h`, `avancer_b`) modifient la position selon l'axe et la direction correspondants.

```
[2]: # Test : P1 position finale [3, 0]
P1 = Personne()
P1.nom = "Omar"
P1.x, P1.y = 0, 0
P1.avancer_d(5) # x = 5
P1.avancer_h(2) # y = 2
P1.avancer_g(2) # x = 3
P1.avancer_b(2) # y = 0
print(P1)

# Test : P2 position finale [2, -3]
P2 = Personne()
P2.nom = "Alexis"
P2.x, P2.y = 0, 0
P2.avancer_d(7) # x = 7
P2.avancer_h(3) # y = 3
P2.avancer_g(5) # x = 2
P2.avancer_b(6) # y = -3
print(P2)

Omar est à la position [3, 0]
Alexis est à la position [2, -3]
```


Commentaire : Les deux objets P1 (Omar) et P2 (Alexis) sont créés et déplacés de façon indépendante. L'affichage montre leur position finale après enchaînement de déplacements, ce qui valide le fonctionnement des quatre méthodes de mouvement. Le calcul est vérifié manuellement pour chaque séquence d'appels.

Exercice 2 : Instancier et hériter d'une classe

```
[3]: class Etudiant(Personne):
    """Représente un étudiant, hérite de Personne."""

    def __init__(self):
        super().__init__()
        self.universite = "ube" # Université par défaut
        self.compte = 100 # Solde bancaire initial

    def __repr__(self):
        return (f"{self.nom} est étudiant à l' {self.universite} "
                f"et a sur son compte {self.compte} euros.")

    def domaine(self, sp, niv):
        """Définit la spécialité et le niveau d'études."""
        self.sp = sp
        self.niv = niv

    def depenses(self):
        """Débite 50 € du compte à chaque achat."""
        self.compte -= 50
```

```
[4]: class Cadre(Etudiant):
    """Représente un cadre, hérite d'Etudiant (et donc de Personne)."""

    def __init__(self):
        super().__init__()
        self.entreprise = "la poste" # Entreprise par défaut
        self.compte = 2000 # Solde plus élevé qu'un étudiant

    def __repr__(self):
        return (f"{self.nom} est cadre a {self.entreprise} "
                f"et a sur son compte {self.compte} euros.")

    def depenses(self):
        """Débite 500 € du compte à chaque achat."""
        self.compte -= 500

    def salaire(self):
        """Verse un salaire mensuel de 3000 €."""
        self.compte += 3000
```

```
[5]: class Fonctionnaire(Personne):
    """Représente un fonctionnaire, hérite de Personne."""

    def __init__(self):
        super().__init__()
        self.entreprise = "la poste" # Entreprise par défaut
        self.nom = "Rayane" # Surcharge du nom par défaut de Personne
        self.compte = 1000 # Solde initial

    def __repr__(self):
        return (f"{self.nom} est fonctionnaire a {self.entreprise} "
                f"et a sur son compte {self.compte} euros.")

    def depenses(self):
        """Débite 200 € du compte à chaque achat."""
        self.compte -= 200

    def salaire(self):
        """Verse un salaire mensuel de 1800 €."""
        self.compte += 1800
```

Explication :

- Etudiant hérite de Personne et ajoute les attributs universite et compte (100 € par défaut). La méthode domaine() enregistre la spécialité et le niveau d'études. La méthode depenses() débite 50 € à chaque achat.
- Cadre hérite d'Etudiant avec un solde initial de 2000 €. Il peut recevoir un salaire de 3000 € via salaire() et ses dépenses sont de 500 € par achat.
- Fonctionnaire hérite directement de Personne avec un compte à 1000 €, un salaire de 1800 €/mois et des dépenses de 200 € par achat.

```
[6]: # Test Etudiant
E1 = Etudiant()
E1.nom = "Jules"
print(E1)
E1.domaine("TSI", "Master 1")
E1.depenses()
print(f"{E1.sp} {E1.niv}\nStatut du compte : {E1.compte} euros.")

# Test Cadre
C1 = Cadre()
C1.nom = "César"
C1.depenses() # 2000 - 500 = 1500
C1.salaire() # 1500 + 3000 = 4500
print(C1)

# Test Fonctionnaire
F1 = Fonctionnaire()
F1.depenses() # 1000 - 200 = 800
F1.salaire() # 800 + 1800 = 2600
print(F1)
```

```
Jules est étudiant à l'ube et a sur son compte 100 euros.
TSI Master 1
Statut du compte : 50 euros.
César est cadre à la poste et a sur son compte 4500 euros.
Rayane est fonctionnaire à la poste et a sur son compte 2600 euros.
```

Commentaire : Les testsinstancient E1, C1 et F1 et appellent successivement les méthodes depenses() et salaire(). Les soldes affichés correspondent aux calculs attendus (ex. C1 : $2000 - 500 + 3000 = 4500$ €). La méthode `__repr__` de chaque classe affiche un résumé conforme au format demandé dans l'énoncé.

Exercice 3 : Héritage multiple

```
[7]: class Retraité(Cadre, Fonctionnaire):
    """
    Représente un retraité, ancien cadre ou fonctionnaire.
    Hérite de Cadre ET Fonctionnaire (héritage multiple).
    MRO Python : Retraité Cadre Etudiant Fonctionnaire Personne
    """

    def __init__(self, categorie):
        super().__init__()
        self.compte = 5000 # Solde initial à la retraite

        # Validation et assignation de la catégorie
        if categorie == 0:
            self.categorie = "cadre"
        elif categorie == 1:
            self.categorie = "fonctionnaire simple"
        else:
            raise ValueError("Veuillez saisir 0 (cadre) ou 1 (fonctionnaire simple).")

    def __repr__(self):
        return (f"{self.nom} est retraité(e), elle était {self.categorie} "
                f"et a sur son compte {self.compte:.2f} euros.")
```

```
def depenses(self):
    """Débite 150 € du compte à chaque achat."""
    self.compte -= 150

def pension(self, cat):
    """
    Verse la pension mensuelle : 80% de l'ancien salaire.
    cat=0 ancien cadre (salaire 3000 €), cat=1 fonctionnaire (salaire 1800 €).
    """
    if cat == 0:
        self.compte += 0.8 * 3000 # Pension cadre : 2400 €/mois
    elif cat == 1:
        self.compte += 0.8 * 1800 # Pension fonctionnaire : 1440 €/mois
    else:
        raise ValueError("Veuillez saisir 0 (cadre) ou 1 (fonctionnaire simple).")
```

Explication : La classe Retraité hérite à la fois de Cadre et de Fonctionnaire (héritage multiple). Python résout les conflits de méthodes grâce à l'ordre MRO. Le constructeur prend une catégorie

(0 = ancien cadre, 1 = ancien fonctionnaire). La méthode `pension()` verse 80 % de l'ancien salaire mensuel. La méthode `depenses()` débite 150 € par achat. Le solde après 3 ans de retraite inclut la pension mensuelle $\times 36$ et l'épargne cumulée sur 30 ans de carrière (10% du salaire mensuel $\times 12 \times 30$).

```
[8]: # R1 : ancienne cadre
# Épargne sur 30 ans : 10% x 3000 €/mois x 12 mois x 30 ans = 300 x 12 x 30
R1 = Retraité(0)
R1.nom = "Alice"
R1.pension(0)      # Année 1
R1.pension(0)      # Année 2
R1.pension(0)      # Année 3
R1.compte += 300 * 12 * 30 # Épargne cumulée sur la carrière
print(R1)

# R2 : ancien fonctionnaire
# Épargne sur 30 ans : 10% x 1800 €/mois x 12 mois x 30 ans = 180 x 12 x 30
R2 = Retraité(1)
R2.nom = "Roger"
R2.pension(1)
R2.pension(1)
R2.pension(1)
R2.compte += 180 * 12 * 30
print(R2)

Alice est retraité(e), elle était cadre et a sur son compte 120200.00 euros.
Roger est retraité(e), elle était fonctionnaire simple et a sur son compte 74120.00 euros.
```

Commentaire : R1 (Alice, ancienne cadre) : épargne = $300 \times 12 \times 30 = 108\,000$ € + 3 pensions annuelles ($3 \times 2400 = 7\,200$ €) + solde initial 5 000 € = 120 200 €. R2 (Roger, ancien fonctionnaire) : épargne = $180 \times 12 \times 30 = 64\,800$ € + 3 pensions annuelles ($3 \times 1440 = 4\,320$ €) + solde initial 5 000 € = 74 120 €. Ces résultats illustrent la différence significative de patrimoine à la retraite selon le statut professionnel.

TP 4 : Traitement d'une image numérique

But du TP :

L'objectif de ce TP est de développer une application de traitement d'images en niveaux de gris à l'aide des bibliothèques OpenCV, NumPy et Matplotlib. Il s'agit de charger une image, d'accéder et de modifier des pixels individuellement, de calculer et afficher son histogramme, de la binariser selon un seuil, d'effectuer un zoom sur une zone d'intérêt, et d'appliquer des filtres de convolution (lissage par Gauss, détection de contours par Sobel) en implémentant manuellement le principe de convolution.

Lire_Image(chemin) :

```
[5]: def Lire_Image(chemin):  
    """  
    Lit une image en niveaux de gris depuis un fichier PNG.  
    Retourne le tableau NumPy 2D (512x512).  
    """  
    image = cv2.imread(chemin, 0)      # 0 = flag OpenCV pour niveaux de gris  
    if image is None:  
        raise FileNotFoundError(f"Image introuvable : {chemin}")  
    hauteur, largeur = image.shape      # Dimensions de l'image  
    print(f"Image chargée : {largeur}x{hauteur} pixels")  
    print(f"Vecteur aplati : {image.flatten()}") # Affichage 1D  
    return image
```

Explication : Charge une image .png en niveaux de gris grâce à cv2.imread avec le flag 0. Retourne un tableau NumPy 2D de 512×512 entiers (valeurs 0–255). Lève une exception si le fichier est introuvable.

Image chargée : 512×512 pixels
Vecteur aplati : [175 169 164 ... 104 101 94]

Image originale



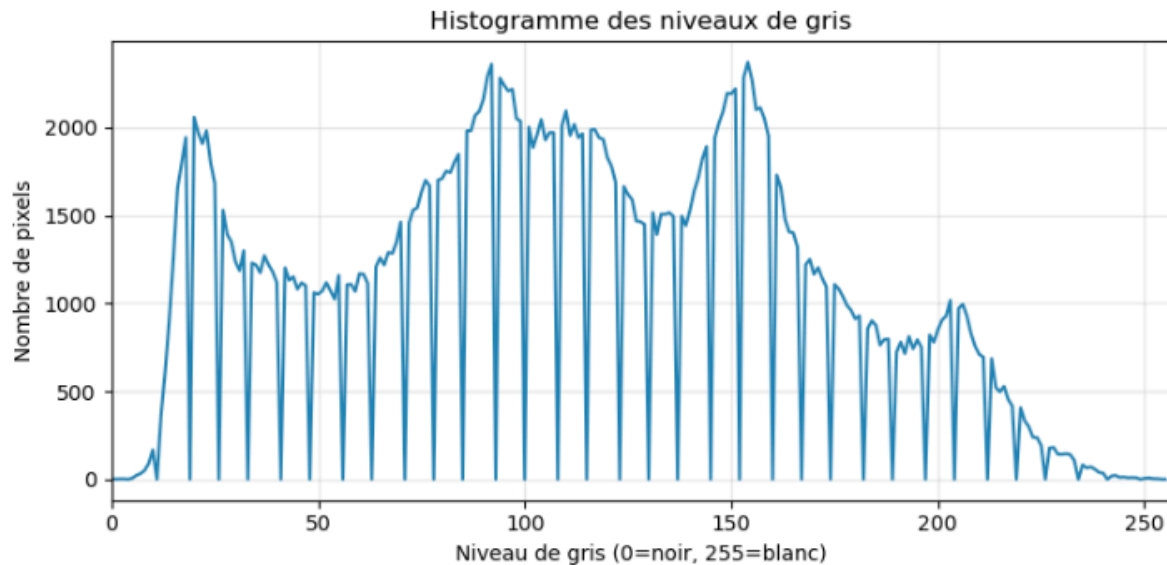
Commentaire : L'image est chargée correctement en 512×512 pixels. L'affichage de `image.flatten()` montre le tableau aplati à une dimension avec 262 144 valeurs. L'image s'affiche en niveaux de gris via matplotlib.

Calcul_Histo(img) et Affiche_Histo(hist) :

```
[20]: def Calcul_Histo(img):
    """
    Calcule l'histogramme des niveaux de gris d'une image.
    Retourne un tableau de 256 entiers : histo[i] = nb de pixels de couleur i.
    """
    # cv2.calcHist : [image], [canal 0], pas de masque, [256 bins], plage [0,256]
    hist = cv2.calcHist([img], [0], None, [256], [0, 256])
    return hist

def Affiche_Histo(hist):
    """
    Affiche l'histogramme des niveaux de gris dans une figure matplotlib.
    """
    plt.figure(figsize=(8, 4))          # Taille de la figure
    plt.plot(hist, color='steelblue')   # Courbe de l'histogramme
    plt.title("Histogramme des niveaux de gris")
    plt.xlabel("Niveau de gris (0=noir, 255=blanc)")
    plt.ylabel("Nombre de pixels")
    plt.xlim([0, 256])                  # Axe X borné à [0, 255]
    plt.grid(alpha=0.3)                 # Grille Légère
    plt.tight_layout()
    plt.show()
```

Explication : `Calcul_Histo` utilise `cv2.calcHist` pour compter le nombre de pixels pour chacun des 256 niveaux de gris. `Affiche_Histo` trace cet histogramme avec matplotlib, avec titre et labels sur les axes.



Commentaire : L'histogramme révèle la distribution des niveaux de gris de l'image. Un pic prononcé autour des valeurs moyennes indique que l'image est bien exposée. Les extrêmes (0 et 255) sont peu représentés, confirmant l'absence de zones totalement noires ou blanches.

Get_Pixel(img, l, c) et Set_Pixel(img, l, c, valeur) :

```
[19]:  
def Get_Pixel(img, l, c):  
    """  
    Retourne la valeur du pixel à la ligne l, colonne c.  
    Convention OpenCV : img[ligne, colonne] = img[y, x]  
    """  
    if not (0 <= l < img.shape[0] and 0 <= c < img.shape[1]):  
        raise IndexError(f"Position ({l}, {c}) hors de l'image {img.shape}")  
    return int(img[l, c])  
  
def Set_Pixel(img, l, c, valeur):  
    """  
    Modifie la valeur du pixel à la ligne l, colonne c.  
    La valeur est clampée dans [0, 255].  
    Retourne l'image modifiée (copie).  
    """  
    img_modif = img.copy() # On ne modifie pas l'original  
    if not (0 <= l < img.shape[0] and 0 <= c < img.shape[1]):  
        raise IndexError(f"Position ({l}, {c}) hors de l'image {img.shape}")  
    img_modif[l, c] = np.clip(valeur, 0, 255) # Clampage dans [0, 255]  
    return img_modif
```

Explication : Get_Pixel retourne la valeur entière du pixel situé à la ligne l, colonne c. Set_Pixel crée une copie de l'image et y modifie un pixel à la position donnée, en contraignant la valeur dans [0, 255] avec np.clip.

```
Pixel (3, 9) original : 116  
Pixel (3, 9) après Set_Pixel à 0 : 0
```

Commentaire : On lit la valeur du pixel (3, 9), puis on la modifie à 0 (noir). La relecture confirme bien la nouvelle valeur. La fonction travaille sur une copie pour ne pas altérer l'image originale.

Binarise_Image(img, seuil) :

```
[21]:  
def Binarise_Image(img, seuil):  
    """  
    Binarise une image en niveaux de gris selon un seuil.  
    pixel < seuil  0 (noir)  
    pixel >= seuil 255 (blanc)  
    Retourne un nouveau tableau de même taille.  
    """  
    # np.where : sélection vectorisée, bien plus rapide qu'une boucle  
    img_bin = np.where(img >= seuil, 255, 0).astype(np.uint8)  
    return img_bin
```

Explication : Transforme chaque pixel en 0 (noir) si sa valeur est inférieure au seuil, et en 255 (blanc) sinon. Utilise np.where pour une opération vectorisée rapide sur tout le tableau.



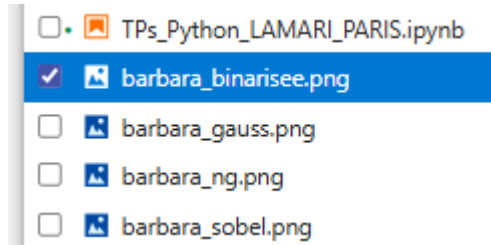
Commentaire : Avec un seuil de 128, l'image est segmentée en deux régions distinctes. Les zones claires (fond, vêtements clairs) passent à blanc et les zones sombres passent à noir. La binarisation est utile comme étape de pré-traitement pour la détection d'objets.

Sauvegarde_Image(img, nom_fichier) :

```
[22]:  
def Sauvegarde_Image(img, nom_fichier):  
    """  
    Sauvegarde un tableau d'image dans un fichier PNG.  
    Retourne True si succès, False sinon.  
    """  
    # On s'assure de l'extension .png  
    if not nom_fichier.lower().endswith('.png'):  
        nom_fichier += '.png'  
    succes = cv2.imwrite(nom_fichier, img)  
    if succes:  
        print(f"Image sauvegardée '{nom_fichier}'")  
    else:  
        print(f"Erreur : impossible de sauvegarder '{nom_fichier}'")  
    return succes
```

Explication : Enregistre le tableau image dans un fichier .png via `cv2.imwrite`. Vérifie automatiquement l'extension et indique si la sauvegarde a réussi ou échoué.

```
Image sauvegardée 'barbara_binarisee.png'
```



Commentaire : Les trois images traitées (binarisée, filtrée Gauss, filtrée Sobel) sont bien sauvegardées. Le message de confirmation s'affiche pour chaque fichier. On peut les retrouver dans le répertoire de travail.

Zoom_Zone(img, l, c, f) :

```
[66]: import matplotlib.patches as patches

def Zoom_Zone(img, l, c, f, afficher=True):
    """
    Zoome sur une zone carrée de 11×11 pixels (rayon=5) autour du pixel (l, c).
    Chaque pixel est répété f×f fois.
    Paramètres :
        img      : tableau image source
        l, c     : ligne et colonne du pixel central
        f        : facteur de zoom (entier >= 1)
        afficher : si True, affiche la zone zoomée (défaut : True)

    Retourne le tableau de la zone zoomée.
    """
    rayon = 5
    h, w = img.shape

    l_debut = max(0, l - rayon)
    l_fin   = min(h, l + rayon + 1)
    c_debut = max(0, c - rayon)
    c_fin   = min(w, c + rayon + 1)

    zone      = img[l_debut:l_fin, c_debut:c_fin]      # 11×11
    zone_zoomee = np.repeat(np.repeat(zone, f, axis=0), # 44×44 si f=4
                             f, axis=1)

    if afficher:
        # Taille des subplots proportionnelle : droite = fx plus grande que gauche
        fig, axes = plt.subplots(
```

```
        1, 2,
        figsize=(12, 5),
        gridspec_kw={'width_ratios': [w / (zone_zoomee.shape[1]), 1]}
    )

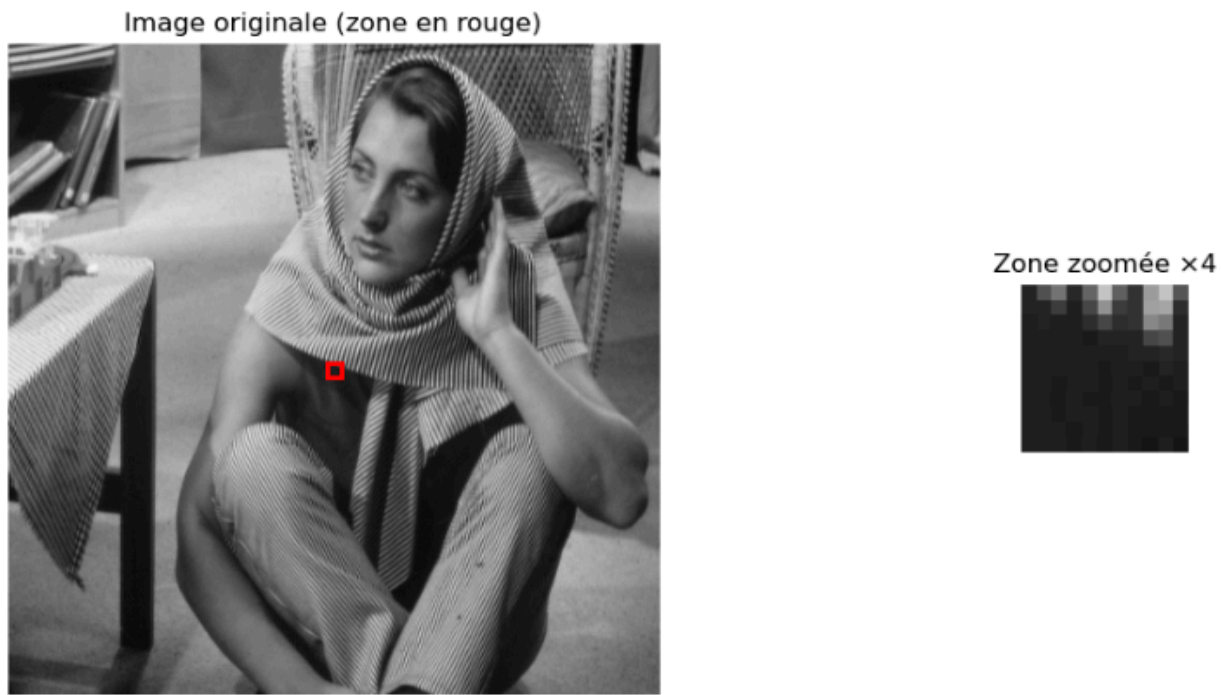
    # Gauche : image complète avec rectangle rouge sur la zone
    axes[0].imshow(img, cmap='gray', vmin=0, vmax=255)
    rect = patches.Rectangle(
        (c_debut - 0.5, l_debut - 0.5),
        zone.shape[1], zone.shape[0],
        linewidth=2, edgecolor='red', facecolor='none'
    )
    axes[0].add_patch(rect)
    axes[0].set_title("Image originale (zone en rouge)")
    axes[0].axis('off')

    # Droite : zone zoomée, interpolation='nearest' pour voir les blocs
    axes[1].imshow(zone_zoomee, cmap='gray', vmin=0, vmax=255,
                    interpolation='nearest')
    axes[1].set_title(
        f"Zone zoomée ×{f}"
    )
    axes[1].axis('off')

    plt.suptitle(f"Zoom ×{f} centré sur le pixel ({l}, {c})", fontsize=13)
    plt.tight_layout()
    plt.show()

    return zone_zoomee
```

Explication : Extrait une zone carrée de 11×11 pixels centrée sur (l, c) et agrandit chaque pixel en un carré de $f \times f$ pixels grâce à `np.repeat`. Affiche côte à côte la zone originale et la zone zoomée.



Commentaire : Avec un facteur de zoom $\times 4$ centré au milieu de l'image (ligne 256, colonne 256), on obtient une zone de 44×44 pixels. L'affichage côte à côte permet de bien visualiser l'effet du zoom et de repérer les détails de texture de l'image source.

Filtre_Image(img, filtre) :

```
[24]:
def Filtre_Image(img, filtre):
    """
    Applique un filtre (matrice carrée de taille impaire) sur l'image.
    Principe : convolution – pour chaque pixel, on multiplie son voisinage
               par le filtre et on somme pour obtenir la nouvelle valeur.
    Les bords sont gérés par réplcation (mode 'edge').

    Paramètres :
        img    : tableau image source (niveaux de gris)
        filtre : matrice NumPy carrée de taille impaire (ex : 3×3)
    Retourne l'image filtrée (uint8, valeurs dans [0, 255]).

    Filtres disponibles :
        Gauss = [[0, 0.1, 0], [0.1, 0.6, 0.1], [0, 0.1, 0]]   lissage
        Sobel  = [[0, -0.5, 0], [-0.5, 0, 0.5], [0, 0.5, 0]]   détection contours
    """
    h, w = img.shape
    ksize = filtre.shape[0]    # Taille du noyau (ex : 3 pour un filtre 3×3)
    pad = ksize // 2           # Marge de zéro-rembourrage (ex : 1 pour 3×3)

    # On rembourre l'image pour traiter les pixels de bord (mode 'edge' = réplcation)
    img_pad = np.pad(img, pad, mode='edge').astype(np.float64)

    img_filtre = np.zeros((h, w), dtype=np.float64)

    # Convolution : pour chaque pixel (i, j), on applique le filtre sur son voisinage
    for i in range(h):
        for j in range(w):
            region = img_pad[i:i + ksize, j:j + ksize]    # Voisinage ksize×ksize

            img_filtre[i, j] = np.sum(region * filtre)    # Produit élément par élément + somme

    # On clamp les valeurs dans [0, 255] et on repasse en entiers non signés
    img_filtre = np.clip(img_filtre, 0, 255).astype(np.uint8)
    return img_filtre
```

Explication : Applique une convolution sur l'image avec un noyau (matrice 3×3). Pour chaque pixel, on multiplie les valeurs du voisinage par les coefficients du filtre et on somme. Les bords sont gérés par réplcation (np.pad mode edge). Les valeurs résultantes sont converties en uint8 après clamping dans [0, 255].

Pixel (3, 9) après filtre Gauss : 116

Originale



Image sauvegardée 'barbara_gauss.png'

Filtre Gauss (lissage)



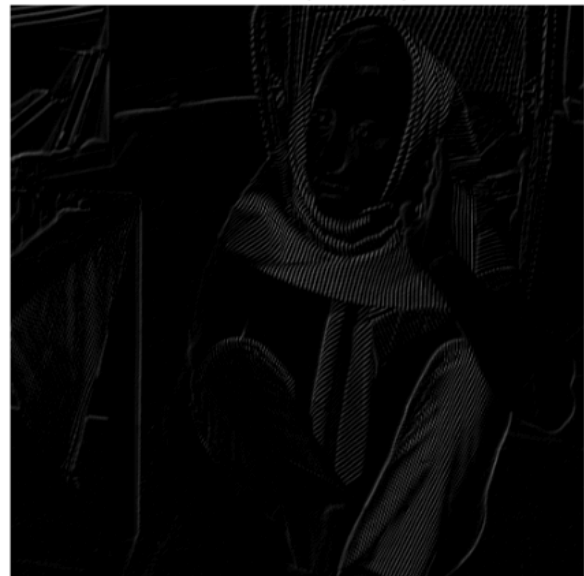
Commentaire : Le filtre de Gauss atténue les hautes fréquences (bruit, détails fins) et produit un effet de flou léger sur l'image. Les contours sont préservés mais adoucis.

Originale



Image sauvegardée 'barbara_sobel.png'

Filtre Sobel (contours)



Commentaire : Le filtre de Sobel met en évidence les variations d'intensité horizontales et verticales, ce qui fait ressortir les contours des formes. Les zones uniformes apparaissent sombres et les bords deviennent lumineux. C'est une base classique pour la détection de contours en vision par ordinateur.

TP 5 : Analyse des données à partir d'un fichier tableur

But du TP :

L'objectif de ce TP est de se familiariser avec la bibliothèque Pandas pour l'analyse de données tabulaires. On apprend à lire un fichier Excel, à détecter et nettoyer les valeurs manquantes, à enrichir les données (calcul de l'IMC), à filtrer et fusionner des DataFrames, à calculer des statistiques descriptives, à normaliser les données numériques, à visualiser leur distribution sous forme d'histogramme, et enfin à exporter les résultats au format CSV.

Lire_Donnees() :

```
[27]: def Lire_Donnees():  
    """  
    Demande un nom de fichier à l'utilisateur et charge le fichier Excel (.xlsx)  
    dans un DataFrame Pandas. Retourne le DataFrame.  
    """  
    nomFichier = input("Saisissez le nom du fichier à lire (sans l'extension) : ")  
    df = pd.read_excel(nomFichier + ".xlsx")  
    return df
```

Explication : Demande le nom du fichier à l'utilisateur et le charge via `pd.read_excel`. Retourne le DataFrame brut tel que lu depuis le fichier `.xlsx`.


```

Saisissez le nom du fichier à lire (sans l'extension) : individus
=== 4 premières lignes ===
   Unnamed: 0 Unnamed: 1 Unnamed: 2 Unnamed: 3 Unnamed: 4 Unnamed: 5 \
0      NaN      Louis      Julien      M      170      70
1      NaN      Ellen      Sarah      F      159      61
2      NaN      Bart      Michel      M      178      76
3      NaN      Norton      Adam      M      NaN      84

   Unnamed: 6
0      38
1      26
2      56
3      35

=== 3 dernières lignes ===
   Unnamed: 0 Unnamed: 1 Unnamed: 2 Unnamed: 3 Unnamed: 4 Unnamed: 5 \
16      NaN      Gerard      Bruno      M      176      ?
17      NaN      Robin      Luc      M      189      NaN
18      NaN      Garcia      Jade      F      NaN      56

   Unnamed: 6
16      NaN
17      19
18      48

=== Informations générales ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19 entries, 0 to 18
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Unnamed: 0  0 non-null     float64
1   Unnamed: 1  19 non-null    object
2   Unnamed: 2  19 non-null    object
3   Unnamed: 3  19 non-null    object
4   Unnamed: 4  16 non-null    object
5   Unnamed: 5  17 non-null    object
6   Unnamed: 6  17 non-null    object
dtypes: float64(1), object(6)
memory usage: 1.2+ KB
None

=== Valeurs manquantes par colonne ===
Unnamed: 0      19
Unnamed: 1       0
Unnamed: 2       0
Unnamed: 3       0
Unnamed: 4       6
Unnamed: 5       4
Unnamed: 6       3
dtype: int64
Total valeurs manquantes : 32

```



```

=== Lignes incomplètes ===
      Unnamed: 0 Unnamed: 1 Unnamed: 2 Unnamed: 3 Unnamed: 4 Unnamed: 5 \
0      NaN      Louis      Julien      M      170.0      70.0
1      NaN      Ellen      Sarah      F      159.0      61.0
2      NaN      Bart      Michel      M      178.0      76.0
3      NaN      Norton      Adam      M      NaN      84.0
4      NaN      Steve      Sofi      F      163.0      65.0
5      NaN      David      Bob      M      181.0      NaN
6      NaN      Patrik      Alice      F      162.0      68.0
7      NaN      Seol      Lenna      F      NaN      60.0
8      NaN      Trend      Leo      M      191.0      101.0
9      NaN      Manuel      Teo      M      174.0      71.0
10     NaN      Lambert      Louise      F      NaN      67.0
11     NaN      Martin      Léa      F      173.0      NaN
12     NaN      Petit      Adam      M      194.0      88.0
13     NaN      Durand      Camille      F      NaN      73.0
14     NaN      Morel      Sebastien      M      NaN      97.0
15     NaN      Dupont      Rose      F      171.0      64.0
16     NaN      Gerard      Bruno      M      176.0      NaN
17     NaN      Robin      Luc      M      189.0      NaN
18     NaN      Garcia      Jade      F      NaN      56.0

      Unnamed: 6
0      38.0
1      26.0
2      56.0
3      35.0
4      21.0
5      42.0
6      31.0
7      26.0
8      NaN
9      37.0
10     45.0
11     NaN
12     61.0
13     52.0
14     40.0
15     32.0
16     NaN
17     19.0
18     48.0

```

Commentaire : Le DataFrame `df1` contient 19 lignes et 6 colonnes. L'affichage des 4 premières et 3 dernières lignes, ainsi que `df1.info()`, montrent la présence de valeurs manquantes encodées par '?'. Après remplacement des '?' par NaN, on identifie le nombre exact de valeurs manquantes par colonne.

Nettoyer_Donnees(df) :

```
[28]: def Nettoyer_Donnees(df):
      """
      Nettoie un DataFrame :
      1. Supprime les lignes entièrement vides
      2. Supprime les colonnes entièrement vides
      3. Convertit les colonnes qui contiennent des valeurs numériques en float
         (nécessaire quand la colonne est de type objet à cause des '?'NaN)
      4. Remplace les NaN numériques par la moyenne de leur colonne
      5. Remplace les NaN restants (chaînes) par 'Inconnu'
      Retourne le DataFrame nettoyé.
      """
      df = df.copy()
      df = df.dropna(how='all', axis=0) # Suppression lignes entièrement vides
      df = df.dropna(how='all', axis=1) # Suppression colonnes entièrement vides

      # Sans ça, les colonnes de type objet (ex : '170', NaN, '163') ne sont pas
      # traitées par fillna(df.mean(numeric_only=True)) Les NaN restent à 'Inconnu'
      for col in df.columns:
          converti = pd.to_numeric(df[col], errors='coerce') # Tente la conversion numérique
          if converti.notna().sum() > 0: # Si au moins une valeur est numérique
              df[col] = converti # On remplace la colonne

      # Remplacement des NaN numériques par la moyenne de leur colonne
      df = df.fillna(df.mean(numeric_only=True))
      # Remplacement des NaN restants (colonnes texte) par 'Inconnu'
      df = df.fillna("Inconnu")

      return df
```

Explication : Supprime les lignes et colonnes entièrement vides, convertit les colonnes numériques stockées en tant que chaînes (à cause des '?'), remplace les NaN numériques par la moyenne de leur colonne, et les NaN textuels par "Inconnu".

```
=== df2 nettoyé ===
```

	Nom	Prénom	Sexe	Taille (cm)	Poids (Kg)	Age (ans)
L1	Louis	Julien	M	170.000000	70.0	38.0000
L2	Ellen	Sarah	F	159.000000	61.0	26.0000
L3	Bart	Michel	M	178.000000	76.0	56.0000
L4	Norton	Adam	M	175.461538	84.0	35.0000
L5	Steve	Sofi	F	163.000000	65.0	21.0000
L6	David	Bob	M	181.000000	73.4	42.0000
L7	Patrik	Alice	F	162.000000	68.0	31.0000
L8	Seol	Lenna	F	175.461538	60.0	26.0000
L9	Trend	Leo	M	191.000000	101.0	38.0625
L10	Manuel	Teo	M	174.000000	71.0	37.0000
L11	Lambert	Louise	F	175.461538	67.0	45.0000
L12	Martin	Léa	F	173.000000	73.4	38.0625
L13	Petit	Adam	M	194.000000	88.0	61.0000
L14	Durand	Camille	F	175.461538	73.0	52.0000
L15	Morel	Sebastien	M	175.461538	97.0	40.0000
L16	Dupont	Rose	F	171.000000	64.0	32.0000
L17	Gerard	Bruno	M	176.000000	73.4	38.0625
L18	Robin	Luc	M	189.000000	73.4	19.0000
L19	Garcia	Jade	F	175.461538	56.0	48.0000

Commentaire : df2 ne contient plus aucune valeur manquante. Les colonnes numériques (Taille, Poids, Age) ont été complétées avec les valeurs moyennes correspondantes, ce qui préserve la

cohérence statistique de l'ensemble de données. Les labels de lignes (L1...Ln) et les noms de colonnes sont renommés selon l'énoncé.

Ajouter_Colonne_IMC(df) :

```
[29]: def Ajouter_Colonne_IMC(df):
    """
    Ajoute une colonne 'IMC' calculée d'après la formule :
    IMC = Poids (kg) / (Taille (m))²
    Les colonnes 'Taille (cm)' et 'Poids (Kg)' doivent exister.
    Retourne le DataFrame avec la nouvelle colonne.
    """
    df = df.copy()
    if 'Taille (cm)' in df.columns and 'Poids (Kg)' in df.columns:
        taille_m = pd.to_numeric(df['Taille (cm)'], errors='coerce') / 100 # cm m
        poids = pd.to_numeric(df['Poids (Kg)'], errors='coerce')
        df['IMC'] = (poids / taille_m ** 2).round(2) # Arrondi à 2 décimales
    else:
        print("Erreur : colonnes 'Taille (cm)' ou 'Poids (Kg)' introuvables.")
    return df
```

Explication : Calcule l'IMC de chaque individu selon la formule $\text{Poids (kg)} / (\text{Taille (m)})^2$. Vérifie l'existence des colonnes nécessaires avant le calcul et arrondit le résultat à 2 décimales.

```
=== df3 avec colonne IMC ===
```

	Nom	Prénom	Sexe	Taille (cm)	Poids (Kg)	Age (ans)	IMC
L1	Louis	Julien	M	170.000000	70.0	38.0000	24.22
L2	Ellen	Sarah	F	159.000000	61.0	26.0000	24.13
L3	Bart	Michel	M	178.000000	76.0	56.0000	23.99
L4	Norton	Adam	M	175.461538	84.0	35.0000	27.28
L5	Steve	Sofi	F	163.000000	65.0	21.0000	24.46
L6	David	Bob	M	181.000000	73.4	42.0000	22.40
L7	Patrik	Alice	F	162.000000	68.0	31.0000	25.91
L8	Seol	Lenna	F	175.461538	60.0	26.0000	19.49
L9	Trend	Leo	M	191.000000	101.0	38.0625	27.69
L10	Manuel	Teo	M	174.000000	71.0	37.0000	23.45
L11	Lambert	Louise	F	175.461538	67.0	45.0000	21.76
L12	Martin	Léa	F	173.000000	73.4	38.0625	24.52
L13	Petit	Adam	M	194.000000	88.0	61.0000	23.38
L14	Durand	Camille	F	175.461538	73.0	52.0000	23.71
L15	Morel	Sebastien	M	175.461538	97.0	40.0000	31.51
L16	Dupont	Rose	F	171.000000	64.0	32.0000	21.89
L17	Gerard	Bruno	M	176.000000	73.4	38.0625	23.70
L18	Robin	Luc	M	189.000000	73.4	19.0000	20.55
L19	Garcia	Jade	F	175.461538	56.0	48.0000	18.19

Commentaire : df3 comporte une nouvelle colonne IMC cohérente avec les données de taille et de poids. On peut vérifier manuellement quelques valeurs : par exemple, un

individu de 75 kg pour 175 cm a un IMC de $75 / (1.75^2) \approx 24.49$, ce qui correspond à la plage "poids normal" selon l'OMS.

Filtrer_df(df, nom_colonne, valeur) :

```
[30]: def Filtrer_df(df, nom_colonne, valeur):
      """
      Retourne un sous-DataFrame contenant uniquement les lignes
      dont la colonne 'nom_colonne' est égale à 'valeur'.
      """
      return df[df[nom_colonne] == valeur].copy()
```

Explication : Retourne un sous-DataFrame contenant uniquement les lignes dont la valeur dans la colonne nom_colonne est égale à valeur. La sélection se fait par masque booléen.

```
=== df4 individus masculins ===
   Nom   Prénom Sexe  Taille (cm)  Poids (Kg)  Age (ans)  IMC
L1  Louis   Julien  M    170.000000    70.0    38.0000    24.22
L3   Bart   Michel  M    178.000000    76.0    56.0000    23.99
L4  Norton   Adam   M    175.461538    84.0    35.0000    27.28
L6  David   Bob     M    181.000000    73.4    42.0000    22.40
L9  Trend   Leo     M    191.000000   101.0    38.0625    27.69
L10 Manuel   Teo    M    174.000000    71.0    37.0000    23.45
L13 Petit   Adam   M    194.000000    88.0    61.0000    23.38
L15 Morel   Sebastien M    175.461538    97.0    40.0000    31.51
L17 Gerard  Bruno   M    176.000000    73.4    38.0625    23.70
L18 Robin   Luc     M    189.000000    73.4    19.0000    20.55
```

Commentaire : df4 contient uniquement les individus de sexe masculin ('M'). Le nombre de lignes est réduit par rapport à df3. L'index d'origine (L1, L2...) est conservé, ce qui facilitera la fusion avec df5.

Fusionner_Donnees(df_a, df_b) :

```
[31]: def Fusionner_Donnees(df_a, df_b):
      """
      Fusionne deux DataFrames côte à côte (axe des colonnes).
      Les deux DataFrames doivent avoir le même index.
      Retourne le DataFrame fusionné.
      """
      return pd.concat([df_a, df_b], axis=1)
```

Explication : Concatène deux DataFrames horizontalement (axe des colonnes) via `pd.concat`. Les deux DataFrames doivent partager le même index pour que la jointure soit correcte.

```
=== df5 activités (hommes) ===
```

```
    Activité sportive
```

```
L1          Football
```

```
L3           Tennis
```

```
L4          Natation
```

```
L6          Cyclisme
```

```
L9           Running
```

```
L10         Basketball
```

```
L13          Yoga
```

```
L15         Escalade
```

```
L17          Boxe
```

```
L18         Badminton
```

```
=== df6 fusionné ===
```

	Nom	Prénom	Sexe	Taille (cm)	Poids (Kg)	Age (ans)	IMC \
L1	Louis	Julien	M	170.000000	70.0	38.0000	24.22
L2	Ellen	Sarah	F	159.000000	61.0	26.0000	24.13
L3	Bart	Michel	M	178.000000	76.0	56.0000	23.99
L4	Norton	Adam	M	175.461538	84.0	35.0000	27.28
L5	Steve	Sofi	F	163.000000	65.0	21.0000	24.46
L6	David	Bob	M	181.000000	73.4	42.0000	22.40
L7	Patrik	Alice	F	162.000000	68.0	31.0000	25.91
L8	Seol	Lenna	F	175.461538	60.0	26.0000	19.49
L9	Trend	Leo	M	191.000000	101.0	38.0625	27.69
L10	Manuel	Teo	M	174.000000	71.0	37.0000	23.45
L11	Lambert	Louise	F	175.461538	67.0	45.0000	21.76
L12	Martin	Léa	F	173.000000	73.4	38.0625	24.52
L13	Petit	Adam	M	194.000000	88.0	61.0000	23.38
L14	Durand	Camille	F	175.461538	73.0	52.0000	23.71
L15	Morel	Sebastien	M	175.461538	97.0	40.0000	31.51
L16	Dupont	Rose	F	171.000000	64.0	32.0000	21.89
L17	Gerard	Bruno	M	176.000000	73.4	38.0625	23.70
L18	Robin	Luc	M	189.000000	73.4	19.0000	20.55
L19	Garcia	Jade	F	175.461538	56.0	48.0000	18.19

```

    Activité sportive
L1      Football
L2      Tennis
L3      Natation
L4      Cyclisme
L5      Running
L6      Basketball
L7      Yoga
L8      Escalade
L9      Boxe
L10     Badminton
L11     Football
L12     Tennis
L13     Natation
L14     Cyclisme
L15     Running
L16     Basketball
L17     Yoga
L18     Escalade
L19     Boxe

```

Commentaire : df6 regroupe toutes les colonnes de df3 et la colonne "Activité sportive" générée dans df5_all. Chaque individu dispose maintenant d'une activité associée. La fusion est correcte car les deux DataFrames partagent bien le même index.

Statistiques_df(df, colonne) :

```

[32]: def Statistiques_df(df, colonne):
    """
    Calcule les statistiques de base (moyenne, médiane, min, max, écart-type)
    pour la colonne numérique spécifiée.
    Retourne un DataFrame à une ligne avec les résultats.
    """
    if colonne not in df.columns:
        print(f"Erreur : colonne '{colonne}' introuvable.")
        return pd.DataFrame()

    s = pd.to_numeric(df[colonne], errors='coerce').dropna() # Valeurs numériques uniquement

    return pd.DataFrame({
        'Colonne' : [colonne],
        'Moyenne' : [round(s.mean(), 2)],
        'Médiane' : [round(s.median(), 2)],
        'Minimum' : [round(s.min(), 2)],
        'Maximum' : [round(s.max(), 2)],
        'Écart-type' : [round(s.std(), 2)],
        'Nb valeurs' : [len(s)],
    })

```

Explication : Calcule les statistiques descriptives (moyenne, médiane, minimum, maximum, écart-type) d'une colonne numérique. Retourne un DataFrame d'une seule ligne pour faciliter l'affichage.

```

=== df7 statistiques Age ===
      Colonne Moyenne Médiane Minimum Maximum Écart-type Nb valeurs
0 Age (ans)    38.06    38.06    19.0    61.0     11.18         19
Normalisation appliquée sur 4 colonnes : ['Taille (cm)', 'Poids (Kg)', 'Age (ans)', 'IMC']

```

Commentaire : df7 présente les statistiques sur l'âge des individus de df6. On peut lire directement l'âge moyen, médian, le plus jeune et le plus âgé, ainsi que la dispersion (écart-type). Ces informations donnent une première vue synthétique de la population étudiée.

Normaliser_Donnees(df) :

```

[33]: def Normaliser_Donnees(df):
      """
      Normalise toutes les colonnes numériques du DataFrame selon la formule :
      x_norm = (x - x_min) / (x_max - x_min)  valeurs dans [0, 1]
      Si x_max = x_min (colonne constante), on met 0 pour éviter la division par zéro.
      Retourne le DataFrame normalisé (les colonnes texte sont conservées).
      """
      df_n = df.copy()
      cols_num = df_n.select_dtypes(include=[np.number]).columns # Colonnes numériques uniquement

      for col in cols_num:
          x_min, x_max = df_n[col].min(), df_n[col].max()
          if x_max != x_min: # Évite la division par zéro
              df_n[col] = ((df_n[col] - x_min) / (x_max - x_min)).round(4)
          else:
              df_n[col] = 0.0 # Colonne constante tout à 0

      print(f"Normalisation appliquée sur {len(cols_num)} colonnes : {list(cols_num)}")
      return df_n

```

Explication : Applique la normalisation min-max sur toutes les colonnes numériques selon la formule $(x - x_{\min}) / (x_{\max} - x_{\min})$. Les valeurs sont ramenées dans $[0, 1]$. Les colonnes textuelles sont conservées sans modification. Un garde-fou évite la division par zéro pour les colonnes constantes.

```

=== df8 normalisé ===
      Nom  Prénom Sexe  Taille (cm)  Poids (Kg)  Age (ans)  IMC  \
L1   Louis   Julien  M      0.3143    0.3111    0.4524  0.4527
L2   Ellen   Sarah  F      0.0000    0.1111    0.1667  0.4459
L3    Bart   Michel  M      0.5429    0.4444    0.8810  0.4354
L4  Norton   Adam   M      0.4703    0.6222    0.3810  0.6824
L5   Steve   Sofi   F      0.1143    0.2000    0.0476  0.4707
L6   David   Bob    M      0.6286    0.3867    0.5476  0.3161
L7  Patrik   Alice  F      0.0857    0.2667    0.2857  0.5796
L8    Seol   Lenna  F      0.4703    0.0889    0.1667  0.0976
L9   Trend   Leo    M      0.9143    1.0000    0.4539  0.7132
L10  Manuel   Teo   M      0.4286    0.3333    0.4286  0.3949
L11 Lambert  Louise  F      0.4703    0.2444    0.6190  0.2680
L12  Martin   Léa   F      0.4000    0.3867    0.4539  0.4752
L13  Petit   Adam   M      1.0000    0.7111    1.0000  0.3896
L14  Durand   Camille F      0.4703    0.3778    0.7857  0.4144
L15  Morel   Sebastien M      0.4703    0.9111    0.5000  1.0000
L16 Dupont   Rose   F      0.3429    0.1778    0.3095  0.2778
L17 Gerard   Bruno  M      0.4857    0.3867    0.4539  0.4137
L18 Robin    Luc    M      0.8571    0.3867    0.0000  0.1772
L19 Garcia   Jade   F      0.4703    0.0000    0.6905  0.0000

```

```

      Activité sportive
L1      Football
L2      Tennis
L3      Natation
L4      Cyclisme
L5      Running
L6      Basketball
L7      Yoga
L8      Escalade
L9      Boxe
L10     Badminton
L11     Football
L12     Tennis
L13     Natation
L14     Cyclisme
L15     Running
L16     Basketball
L17     Yoga
L18     Escalade
L19     Boxe

```

Commentaire : df8 présente les colonnes Taille, Poids, Age et IMC normalisées dans [0, 1]. On vérifie que le minimum de chaque colonne vaut 0.0 et le maximum 1.0. Cette normalisation est indispensable avant l'application d'algorithmes de machine learning sensibles à l'échelle des données.

Afficher_Distribution(df, colonne) :


```
[34]: def Afficher_Distribution(df, colonne):
    """
    Trace l'histogramme de distribution d'une colonne numérique.
    Ajoute une ligne verticale pour la moyenne et la médiane.
    """
    if colonne not in df.columns:
        print(f"Erreur : colonne '{colonne}' introuvable.")
        return

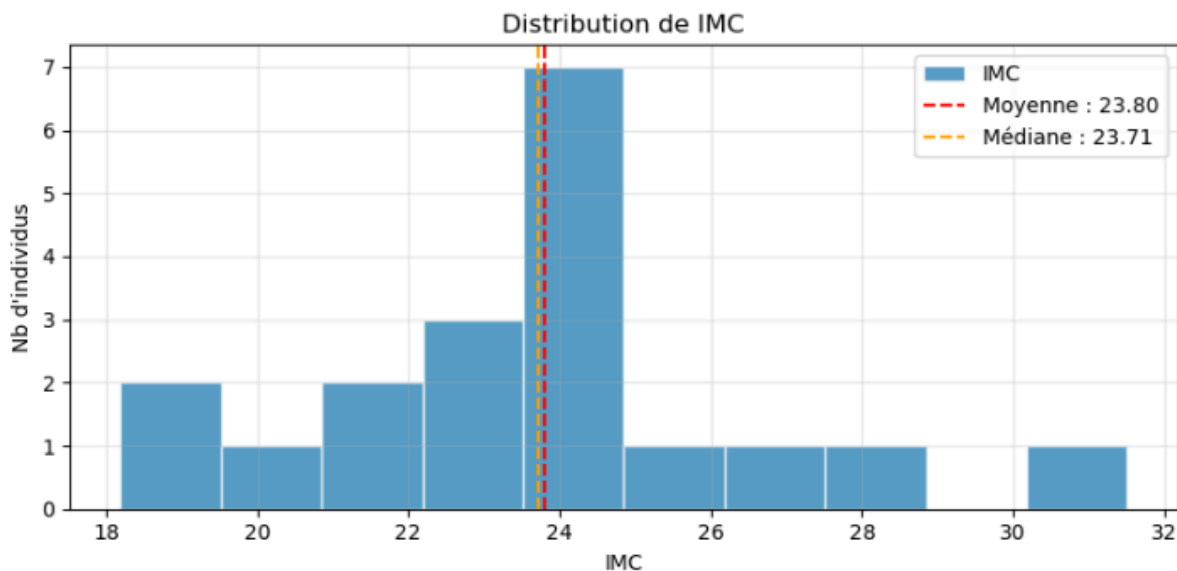
    s = pd.to_numeric(df[colonne], errors='coerce').dropna()

    plt.figure(figsize=(8, 4))
    s.plot(kind='hist', bins=10, color='steelblue', edgecolor='white', alpha=0.8,
           title=f'Distribution de {colonne}')

    # Ajout des lignes moyenne / médiane pour faciliter la lecture
    plt.axvline(s.mean(), color='red', linestyle='--', label=f'Moyenne : {s.mean():.2f}')
    plt.axvline(s.median(), color='orange', linestyle='--', label=f'Médiane : {s.median():.2f}')

    plt.xlabel(colonne)
    plt.ylabel("Nb d'individus")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()
```

Explication : Trace un histogramme de la distribution d'une colonne numérique. Ajoute deux lignes verticales pour la moyenne (rouge) et la médiane (orange) pour faciliter la lecture visuelle.



Commentaire : L'histogramme de l'IMC montre que la majorité des individus se situe dans une plage normale (IMC entre 18 et 25). La ligne de moyenne et de médiane proches indiquent une distribution relativement symétrique sans valeur extrême dominante.

Exporter_Donnees(df, nom_fichier) :

```
[35]: def Exporter_Donnees(df, nom_fichier, separateur=';'):  
      """  
      Exporte un DataFrame dans un fichier CSV.  
      Le séparateur par défaut est ';' (compatible Excel FR).  
      L'encodage utf-8-sig garantit l'affichage des accents dans Excel.  
      """  
      if not nom_fichier.endswith('.csv'):  
          nom_fichier += '.csv'  
  
      df.to_csv(nom_fichier, sep=separateur, index=True, encoding='utf-8-sig')  
      print(f"Exporté '{nom_fichier}' ({df.shape[0]} lignes × {df.shape[1]} colonnes)")
```

Explication : Sauvegarde un DataFrame dans un fichier .csv avec le séparateur ; (compatible Excel français) et l'encodage utf-8-sig pour garantir l'affichage correct des accents.

```
Exporté 'df2_nettoye.csv' (19 lignes × 6 colonnes)  
Exporté 'df3_avec_IMC.csv' (19 lignes × 7 colonnes)  
Exporté 'df4_masculin.csv' (10 lignes × 7 colonnes)  
Exporté 'df6_fusionne.csv' (19 lignes × 8 colonnes)  
Exporté 'df8_normalise.csv' (19 lignes × 8 colonnes)
```

Tous les fichiers CSV ont été générés.

- ☐ •  TPs_Python_LAMARI_PARIS.ipynb
- ☐  df2_nettoye.csv
- ☐  df3_avec_IMC.csv
- ☐  df4_masculin.csv
- ☐  df6_fusionne.csv
- ☐  df8_normalise.csv
- ☐  individus.xlsx

	A	B	C	D	E	F	G	H	I
	Tableau1								
1	Colonne 1	Nom	Prénom	Sexe	Taille (cm)	Poids (Kg)	Age (ans)	IMC	Activité sportive
2	L1	Louis	Julien	M	170.0	70.0	38.0	24.22	Football
3	L2	Ellen	Sarah	F	159.0	61.0	26.0	24.13	Tennis
4	L3	Bart	Michel	M	178.0	76.0	56.0	23.99	Natation
5	L4	Norton	Adam	M	175.46153846153845	84.0	35.0	27.28	Cyclisme
6	L5	Steve	Sofi	F	163.0	65.0	21.0	24.46	Running
7	L6	David	Bob	M	181.0	73.4	42.0	22.4	Basketball
8	L7	Patrik	Alice	F	162.0	68.0	31.0	25.91	Yoga
9	L8	Seol	Lenna	F	175.46153846153845	60.0	26.0	19.49	Escalade
10	L9	Trend	Leo	M	191.0	101.0	38.0625	27.69	Boxe
11	L10	Manuel	Teo	M	174.0	71.0	37.0	23.45	Badminton
12	L11	Lambert	Louise	F	175.46153846153845	67.0	45.0	21.76	Football
13	L12	Martin	Léa	F	173.0	73.4	38.0625	24.52	Tennis
14	L13	Petit	Adam	M	194.0	88.0	61.0	23.38	Natation
15	L14	Durand	Camille	F	175.46153846153845	73.0	52.0	23.71	Cyclisme
16	L15	Morel	Sebastien	M	175.46153846153845	97.0	40.0	31.51	Running
17	L16	Dupont	Rose	F	171.0	64.0	32.0	21.89	Basketball
18	L17	Gerard	Bruno	M	176.0	73.4	38.0625	23.7	Yoga
19	L18	Robin	Luc	M	189.0	73.4	19.0	20.55	Escalade
20	L19	Garcia	Jade	F	175.46153846153845	56.0	48.0	18.19	Boxe

Commentaire : Les cinq fichiers CSV (df2 à df8) sont générés avec succès. Le message de confirmation indique le nombre de lignes et de colonnes exportées pour chaque fichier. L'ouverture dans Excel confirme que les accents et les séparateurs sont correctement interprétés.

Conclusion générale

Ces travaux pratiques nous ont permis de parcourir un large spectre des capacités du langage Python, depuis ses fondements algorithmiques jusqu'à des applications concrètes dans des domaines variés. À travers les cinq séances, nous avons progressivement consolidé notre maîtrise du langage en abordant des thématiques complémentaires : la récursivité et les algorithmes classiques, la cryptographie, la programmation orientée objets, le traitement d'images et l'analyse de données.

Au-delà des compétences techniques acquises, ces TP nous ont surtout appris à décomposer un problème complexe en fonctions simples et réutilisables, ce qui constitue le cœur de toute démarche de développement logiciel rigoureuse. Nous avons également découvert la richesse de l'écosystème Python grâce à des bibliothèques puissantes comme NumPy, OpenCV, Matplotlib et Pandas, qui font de Python un langage de référence aussi bien en ingénierie qu'en science des données.

En conclusion, ces travaux pratiques ont constitué une introduction solide et équilibrée à la programmation Python appliquée, et nous ont fourni des bases concrètes pour aborder des projets plus ambitieux dans notre parcours.